Moderní symetrické blokové šifry

STUDIJNÍ SKRIPTA · 7UDZK · PŘEDNÁŠKA 2

Samostatný výklad podle prezentace **7UDZK-2.pdf**

Jak funguje AES uvnitř, proč samotná bloková šifra nestačí a jak se liší režimy ECB, CBC, CTR, GCM a XTS.

> **Cíl skript:** umět vysvětlit transformaci jednoho bloku, projít základní výpočty v AES a rozpoznat, jak podmínky použití režimu ovlivňují důvěrnost, integritu i výkon celého systému.

Podklad: Mgr. Ondřej Surý, University of Ostrava. Dodaná prezentace obsahuje 21 snímků. Její témata tvoří kostru těchto skript; výklad, výpočty a procvičení jsou nově zpracovány pro samostudium.

Návaznost: předpokládá se základní představa o klíči, šifrování, XOR a autentizaci. Potřebné značení se znovu vysvětluje. Konečná tělesa se zavádějí od konkrétního bajtu, takže není nutné předem znát abstraktní algebru.

Odborná kontrola: parametry a operace AES vycházejí ze standardu FIPS 197. U režimů rozlišujeme požadavky na IV, důsledky změn při šifrování a při dešifrování a rozdíl mezi šifrováním a autentizací.

Zpracováno 15. října 2026.

Jak se skripty pracovat

ORIENTACE A OBSAH

Kapitoly 1-12 vysvětlují konstrukci blokových šifer a AES. Kapitoly 13-22 ukazují, jak šifru použít na skutečná data. Následují útoky, další algoritmy, opravy původního podkladu a procvičení. **[P]** označuje prezentaci; číslované odkazy odkazují na zdroje na konci.

1. Co je symetrická bloková šifra	3
2. Konfúze, difúze a lavinový efekt	4
3. DES a proč ho nezachránil krátký klíč	5
4. Feistelova síť: vratnost bez inverze F	6
5. AES: historie a základní parametry	7
6. Bajty, hexadecimální zápis a stav AES	8
7. GF(2^8): proč se s bajty počítá jinak	9
8. Celý průběh šifrování AES	10
9. SubBytes a ShiftRows krok za krokem	11
10. MixColumns: vypočítejte jeden sloupec	12
11. Podklíče, AddRoundKey a dešifrování	13
12. Ověřovací příklad celého AES-128	14
13. Režimy použití a doplnění zprávy	15
14. ECB: proč mohou zůstat vidět vzory	16
15. CBC: řetězení šifrových bloků	17
16. Chyby CBC a rozdíl proti CBC-MAC	18
17. CFB, OFB a PCBC v jednom přehledu	19
18. CTR: ze šifry vznikne klíčový proud	20
19. IV, nonce a srovnání režimů	21
20. GCM: čítač a autentizace společně	22
21. AEAD v protokolu a v aplikaci	23
22. XTS: proč disk není síťová zpráva	24
23. Co přesně znamená útok na AES	25
24. Implementace, kvantové útoky a člověk	26
25. Serpent, Twofish a další finalisté	27
26. Camellia a evropský projekt NESSIE	28
27. Co v prezentaci upřesnit	29
Procvičení, řešení, slovníček a zdroje	30-37

Dvě úrovně podrobnosti. Pro porozumění principům stačí znát účel operací AES a umět číst vzorce režimů. Pro hlubší zvládnutí projděte výpočty v GF(2^8), první rundu AES a příklady útoků. Tyto výpočty ukazují mechanismus; nejsou návodem psát vlastní produkční kryptografickou knihovnu.

Důležitá hranice: samotná volba algoritmu, například AES-256, neurčuje bezpečnost celého řešení. Záleží také na režimu, klíči, nonce, autentizaci, implementaci a protokolu.

1. Co je symetrická bloková šifra

VAZBA NA PREZENTACI: SNÍMEK 3

Symetrická šifra používá sdílený tajný klíč. Oprávněné strany znají stejné základní tajemství pro šifrování i dešifrování; interní rundové podklíče a postup výpočtu se přitom mohou lišit. Algoritmus může být veřejný. Ochrana stojí na neznámém klíči a odolné konstrukci.

Bloková šifra zpracuje blok pevné délky n bitů a vrátí blok stejné délky. Pro pevný klíč je transformace permutací všech 2^n možných bloků. Musí tedy být oboustranně jednoznačná: dva různé vstupní bloky se pro stejný klíč nesmějí slít do stejného výsledku.

```
C = E_K(P)
P = D_K(C)
D_K(E_K(P)) = P
```

P znamená otevřený blok, C šifrový blok, K klíč. E a D jsou dopředná a inverzní transformace. Korektnost poslední rovnosti neprokazuje bezpečnost: i Caesarův posun lze bezchybně obrátit, ale útočník snadno vyzkouší všechny klíče.

Délka bloku není délka klíče

AES má blok **128 bitů = 16 bajtů**, zatímco klíč má 128, 192 nebo 256 bitů. Varianta AES-256 tedy stále zpracovává 16bajtové bloky. Delší klíč zvětšuje prostor kandidátních klíčů, nezvětšuje automaticky blok ani počet bajtů jedné základní transformace.

U DES je blok 64 bitů = 8 bajtů a efektivní klíč 56 bitů. U dalších šifer mohou být jiné parametry. Jakmile je však zvolena konkrétní šifra, například AES, implementace si její délku bloku nemůže svévolně měnit.

Proč to ještě není hotové šifrování souboru

Zpráva může mít jeden bajt, několik bloků i gigabajty. Samotný předpis E_K říká, jak převést jeden blok. Neřeší, jak zpracovat různou délku zpráv, jak skrýt opakování bloků ani jak odhalit podvržení. To přidává **režim použití** a případně další protokol.

> **Tři oddělené pojmy:** bloková šifra je základní transformace; režim určuje její použití na zprávu; protokol navíc řeší například klíče, identitu protistrany a pořadí zpráv.

2. Konfúze, difúze a lavinový efekt

VAZBA NA PREZENTACI: SNÍMEK 3

Proč nestačí přeskupit písmena

Přímočaré transformace často zachovávají vztahy, které útočník využije. Klasická substituce mění názvy symbolů, ale může zachovat jejich četnosti. Moderní šifra musí mnohem důkladněji skrýt využitelné vztahy mezi vstupem, klíčem a výstupem.

Konfúze znamená zastření vztahu mezi klíčem a šifrovým textem. Útočník nemá ze snadné vlastnosti výsledku přímo vyčíst část klíče. Důležitou roli hraje nelinearita. V AES ji dodává zejména substituce SubBytes pomocí S-boxu.

Difúze rozptyluje vliv částí vstupu do více částí výstupu. V AES přesouvá ShiftRows bajty mezi sloupci a MixColumns kombinuje bajty uvnitř sloupce. Jejich střídáním se vliv původně místní změny šíří stavem.

Lavinový efekt není přesná rovnost

U dobré šifry očekáváme, že změna jednoho vstupního bitu při pevném klíči povede po celém šifrování v průměru ke změně přibližně poloviny výstupních bitů. U 128bitového bloku tedy zhruba 64. Neznamená to, že každý jednotlivý experiment musí změnit právě 64 bitů.

Pro dvě nezávisle rovnoměrné 128bitové hodnoty má počet odlišných bitů očekávanou hodnotu $128 \cdot 1/2 = 64$ a směrodatnou odchylku $\sqrt{128 \cdot 1/2 \cdot 1/2} \approx 5{,}66$. Jde o vysvětlující referenční model, nikoli důkaz, že se libovolná implementace AES přesně takto chová.

Proč se používá více rund

Jedna vrstva substituce mění bajty místně. Jedna permutace jen mění jejich pozice. Teprve vhodné opakování nelineárních a rozptylujících kroků rozvine závislosti po celém bloku. Klíč se přitom průběžně přimíchává pomocí podklíčů.

Runda neboli kolo je přesně definovaná posloupnost operací. Její počet nelze oddělit od konstrukce. Dvacet rund jedné šifry nemusí být bezpečnějších než deset rund jiné: rundy mohou dělat zcela odlišnou práci.

> **Omezení intuice:** silný lavinový efekt je žádoucí vlastnost, ale sám bezpečnost nedokazuje. Může existovat jiná algebraická slabina, slabá správa klíče nebo nebezpečný režim použití.

3. DES a proč ho nezachránil krátký klíč

VAZBA NA PREZENTACI: SNÍMEK 4

Data Encryption Standard (DES) byl standardizován v roce 1977 tehdejším NBS, předchůdcem NIST. Vycházel z návrhů IBM a souvisel s vývojem šifry Lucifer; do posouzení a úprav vstupovala i NSA. Má 64bitový blok, 56 efektivních bitů klíče a 16 Feistelových rund.

Klíč bývá uložen v 64 bitech, ale osm bitů tvoří paritní bity. Prostor tajných možností je proto 2^{56}, nikoli 2^{64}. Zvětšení obalu klíče nepřidává tajemství, pokud některé bity určují kontrolní pravidla.

Hrubá síla se stala praktickou

Stroj **Deep Crack** organizace EFF v roce 1998 vyřešil jednu výzvu DES za přibližně 56 hodin. Spolupráce s distributed.net v roce 1999 zkrátila další výzvu na 22 hodin a 15 minut. Jde o konkrétní experimenty hledání klíče, nikoli pevnou univerzální dobu každého útoku. [3]

Hlavní praktickou slabinou je malý prostor klíčů. Formulace „krátký klíč, ne návrh“ je ale zkratka: existují i kryptoanalytické metody a 64bitový blok omezuje bezpečné objemy dat v řadě režimů. Parametry a režim musí dávat smysl společně.

Triple DES neboli TDEA

Triple DES aplikuje DES třikrát, typicky ve schématu EDE:

```
C = E_K3(D_K2(E_K1(P)))
```

Písmeno D uprostřed znamená dešifrovací transformaci DES. Pokud jsou všechny tři klíče stejné, prostřední dvě operace se vyruší a zůstane jediný DES. To umožňovalo zpětnou kompatibilitu. Při různých klíčích nejde prostě o „DES s třikrát delším blokem“: blok zůstává 64bitový.

Tříklíčové TDEA má 168 efektivních klíčových bitů, ale jeho obecná bezpečnost se nehodnotí jako 168bitová; běžně se uvádí úroveň přibližně 112 bitů. Dvouklíčová varianta vyžaduje vlastní, odlišné posouzení.

NIST od 1. ledna 2024 stáhl SP 800-67 Rev. 2 a nepovoluje TDEA pro novou ochranu dat ve svém rámci. Některé operace se starými daty, například jejich dešifrování, mají odlišný přechodový režim. Nejde o obecný právní zákaz platný pro každého uživatele. [4]

> **Poučení:** životnost algoritmu závisí na parametrech, útocích i objemu dat. Zastaralý blok nezvětšíme tím, že jeho transformaci několikrát zopakujeme.

4. Feistelova síť: vratnost bez inverze F

VAZBA NA PREZENTACI: SNÍMEK 5

Feistelova konstrukce dělí blok na dvě poloviny L a R. V každé rundě transformuje jednu polovinu pomocí funkce F a podklíče K_i, výsledek XORuje s druhou polovinou a role prohodí:

```
L_(i+1) = R_i
R_(i+1) = L_i ⊕ F(R_i, K_i)
```

Index i označuje pořadí rundy; nejde o mocninu. Znak ⊕ je XOR. Pro bity platí 0 ⊕ 0 = 0, 0 ⊕ 1 = 1, 1 ⊕ 0 = 1 a 1 ⊕ 1 = 0. Dvojí XOR se stejnou hodnotou se vyruší.

Jak kolo obrátíme

Z první rovnosti ihned známe původní R_i. Pak dopočítáme původní L_i:

```
R_i = L_(i+1)
L_i = R_(i+1) ⊕ F(L_(i+1), K_i)
```

Nepotřebovali jsme počítat F^{-1}. Stačí znovu vyhodnotit F na známém vstupu a XORem odstranit stejnou hodnotu. **F nemusí být prostá ani samostatně vratná**, a přesto je celá Feistelova runda vratná.

Číselný příklad

Použijeme hračkový 8bitový blok rozdělený na dvě 4bitové poloviny. Zvolíme L = A_{16} = 10, R = 3 a podklíč 5. Jen pro ukázku definujeme F(R,K) = (R + K) mod 16. Výsledek F je 8.

Krok	Výpočet	Výsledek
Nová levá polovina	L' = R	3
Nová pravá polovina	R' = A_{16} ⊕ 8_{16}	2
Obnova staré pravé	R = L'	3
Obnova staré levé	L = 2_{16} ⊕ 8_{16}	A_{16}

Taková ukázková F nemá poskytovat skutečnou bezpečnost. Slouží pouze k ověření vratnosti.

Dešifrování celé sítě

Podklíče se zpracovávají v opačném pořadí. Po zohlednění konvence závěrečného prohození polovin může šifrování i dešifrování využít stejnou strukturu. DES, Blowfish a Camellia používají Feistelovské konstrukce; Twofish používá příbuznou konstrukci s dalšími operacemi.

> **Vratnost není bezpečnost.** I F, která vždy vrátí nulu, poskytne vratnou rundu: jen prohodí poloviny. Kvalita F a počet rund rozhodují o tom, zda síť chrání obsah.

5. AES: historie a základní parametry

VAZBA NA PREZENTACI: SNÍMKY 6-7 A 16

NIST zahájil výběr nového standardu v roce 1997. Z patnácti přijatých kandidátů postoupily do finále **MARS, RC6, Rijndael, Serpent a Twofish**. Dne 2. října 2000 oznámil NIST výběr Rijndaelu; standard FIPS 197 byl vydán 26. listopadu 2001 a nabyl účinnosti 26. května 2002. [1, 2]

Autory Rijndaelu jsou **Joan Daemen a Vincent Rijmen**. AES je standardizovaná část širší rodiny Rijndael. U AES je blok vždy 128 bitů, i když širší rodina umožňovala další varianty. Aktualizace FIPS 197 z roku 2023 zpřehlednila popis bez technické změny algoritmu. [1]

Varianta	Tajný klíč	Blok	Rundy N_r	128bitové podklíče
AES-128	16 bajtů	16 bajtů	10	11
AES-192	24 bajtů	16 bajtů	12	13
AES-256	32 bajtů	16 bajtů	14	15

Počet podklíčů je o jeden vyšší než počet rund: jeden se přidává před první rundou. Základní klíč se neposílá do každého kola nezávisle v nové náhodné podobě; všechny podklíče deterministicky odvozuje **expanze klíče**.

Proč zvítězil Rijndael

Posuzovala se bezpečnost, cena a výkon i vlastnosti implementace. Záleželo na různých procesorech, paměťových nárocích, hardwaru a jednoduchosti. Výběr proto není důkazem, že každý jiný finalista je prolomený nebo že Rijndael zvítězil jediným univerzálním měřítkem.

Počty kladných a záporných hlasů na snímku 16 nelze bez uvedení přesného původu a metodiky chápat jako oficiální počet bezpečnostních bodů. V těchto skriptech se o ně neopíráme; rozhodující je veřejný hodnoticí proces a zdůvodnění NIST. [2]

Co znamená standard v praxi

Shodný klíč a shodný blok mají ve správných implementacích AES vytvářet shodný výstup. Platforma může měnit rychlost nebo způsob provedení, nikoli předepsanou matematickou transformaci. AES samo neobsahuje nonce, padding ani autentizační tag: ty přidává režim.

> **Bezpečnost bez data spotřeby?** To, že standard nemá předem dané datum ukončení, není slib neomezené životnosti. Algoritmus a podmínky použití se průběžně odborně přehodnocují.

6. Bajty, hexadecimální zápis a stav AES

VAZBA NA PREZENTACI: SNÍMEK 7

Bit je 0 nebo 1. **Bajt** má osm bitů a 256 možných hodnot. Hexadecimální zápis používá šestnáct číslic 0-9 a A-F, takže jeden bajt zapíšeme dvěma hexadecimálními číslicemi. Hodnota 53_{16} například znamená $5 \cdot 16 + 3 = 83$ v desítkové soustavě.

Ve výpočtech dále zápis `0x53` výslovně označuje hexadecimální hodnotu. Dvojice `53` v tabulkách a v blocích AES také čteme hexadecimálně, není-li uvedeno jinak. Předpona 0x není součástí uloženého bajtu.

Blok jako matice

AES ukládá 16 bajtů do **stavu**, matice 4 × 4. Plní ji **po sloupcích**, nikoli po řádcích. Pro vstupní bajty b0 až b15 v pořadí přenosu vypadá stav takto:

Řádek	Sloupec 0	Sloupec 1	Sloupec 2	Sloupec 3
0	b0	b4	b8	b12
1	b1	b5	b9	b13
2	b2	b6	b10	b14
3	b3	b7	b11	b15

Formálně s[r,c] = vstup[4c + r], kde r je řádek a c sloupec. Pro blok `00112233445566778899aabbccddeeff` dostaneme:

Řádek	Sloupec 0	Sloupec 1	Sloupec 2	Sloupec 3
0	00	44	88	cc
1	11	55	99	dd
2	22	66	aa	ee
3	33	77	bb	ff

Po skončení se stav opět čte po sloupcích. Jde o dva pohledy na stejných 128 bitů, nikoli o rozšíření nebo kompresi zprávy.

Proč na uspořádání záleží

ShiftRows posouvá řádky této konkrétní matice. MixColumns kombinuje její sloupce. Kdo naplní stav po řádcích, může provést jednotlivé operace zdánlivě správně, ale získá jiný algoritmus a neprojde ověřovacím příkladem.

> **Kontrolní otázka:** kam patří osmý bajt vstupu, počítáme-li od jedničky? Je to b7, tedy řádek 3 a sloupec 1. Indexy začínají nulou. Toto rozlišení ušetří většinu zmatků při ručním sledování AES.

7. GF(2⁸): proč se s bajty počítá jinak

ROZVEDENÍ MATEMATIKY ZE SNÍMKU 7

Konečné těleso GF(2⁸) má 256 prvků. Každý reprezentujeme osmi bity b7…b0 a současně polynomem $b7 \cdot x^7 + \ldots + b1 \cdot x + b0$. Koeficienty jsou jen 0 a 1 a počítá se s nimi modulo 2. Bajt 0x53 = 01010011 tedy představuje $x^6 + x^4 + x + 1$.

Sčítání je XOR

Při sčítání koeficientů modulo 2 se 1 + 1 rovná 0, bez přenosu do vyšší pozice. Proto součet dvou prvků tělesa odpovídá XOR jejich bitů. Například 0x57 ⊕ 0x83 = **0xd4**. Stejná operace funguje jako odčítání.

Není to obyčejné sčítání bajtů modulo 256: tam bychom dostali 0x57 + 0x83 = 0xda. Tyto dvě aritmetiky nesměšujeme.

Násobení a redukce

Polynomy vynásobíme a výsledek zredukujeme modulo pevný nerozložitelný polynom AES **$m(x) = x^8 + x^4 + x^3 + x + 1$**, reprezentovaný hodnotou 0x11b. Díky redukci výsledek opět reprezentuje jediný bajt.

Často potřebujeme násobení hodnotou 0x02, tedy polynomem x. Operace **xtime(a)** posune bity vlevo; pokud měl původní bajt nejvyšší bit 1, po omezení na osm bitů navíc provede XOR s 0x1b.

Příklad	Posun a omezení na bajt	Redukce	Výsledek
0x02 · 0x57	0xae	Není třeba	0xae
0x02 · 0x83	0x06	0x06 ⊕ 0x1b	0x1d
0x03 · 0x57	xtime(0x57) ⊕ 0x57	0xae ⊕ 0x57	0xf9

Protože 0x03 = x + 1, násobení trojkou v této reprezentaci znamená xtime(a) ⊕ a. Nejde o obyčejné trojnásobení čísla s následným zbytkem po dělení 256.

Co znamená „těleso“

Každý nenulový prvek má multiplikativní inverzi: existuje a^{-1} takové, že $a \cdot a^{-1} = 1$. Nula inverzi nemá. Tuto vlastnost využívá konstrukce S-boxu. MixColumns používá stejné těleso pro násobení bajtů pevnými koeficienty.

> **Stačí si odnést:** součet = XOR; násobení = polynomy s redukcí; výsledek zůstane bajtem. Teorie vysvětluje operace, nepožaduje ruční násobení všech 256 možných hodnot.

8. Celý průběh šifrování AES

VAZBA NA PREZENTACI: SNÍMEK 8

AES je **substitučně-permutační konstrukce**, nikoli klasická Feistelova síť. Základní operace působí na celý stav, byť některé zpracovávají jednotlivé bajty a jiné řádky či sloupce. Algoritmus není pouhé opakované zašifrování stejného bloku nezávislými šiframi.

Tři části výpočtu

Část	Operace v pořadí	Kolikrát pro AES-128
Příprava	Expanze základního klíče na podklíče	Jednou pro daný klíč
Počátek	AddRoundKey s podklíčem K0	Jednou
Běžná runda	SubBytes → ShiftRows → MixColumns → AddRoundKey	Devětkrát
Poslední runda	SubBytes → ShiftRows → AddRoundKey	Jednou

AES-192 má jedenáct běžných rund a jednu poslední; AES-256 třináct běžných a jednu poslední. Počáteční AddRoundKey není další plná runda. Proto standard uvádí 10, 12 a 14 rund, nikoli 11, 13 a 15.

Co dělají čtyři transformace

SubBytes nahradí každý bajt podle jedné pevné nelineární tabulky. **ShiftRows** přesune bajty cyklickým posunem jednotlivých řádků. **MixColumns** spojí čtyři bajty každého sloupce pomocí lineární transformace nad $GF(2^8)$. **AddRoundKey** XORuje stav s příslušným 16bajtovým podklíčem.

Každá operace má jiný účel. Pouze AddRoundKey přímo přimíchává klíč do stavu. Zbývající tři jsou veřejně definované transformace, jejichž účinek se skládá s klíčovým krokem.

Proč poslední runda nemá MixColumns

Vynechání je součást specifikace, ne neúplnost obrázku. Poslední MixColumns samo o sobě představuje veřejnou vratnou lineární transformaci; jeho pouhé přidání za hotový výstup by nevytvořilo nové tajemství. Úplný bezpečnostní rozbor je složitější, ale pro čtení algoritmu je podstatné poslední rundu skutečně odlišit.

Proč se dá AES dešifrovat

SubBytes je permutace bajtů, ShiftRows permutace pozic, MixColumns invertibilní lineární transformace a XOR podklíče se obrací opět XORem. Každý krok má inverzi, proto můžeme celou jejich posloupnost obrátit.

> **Kontrola počtů:** u AES-128 proběhne SubBytes desetkrát, ShiftRows desetkrát, MixColumns devětkrát a AddRoundKey jedenáctkrát. Klíčový rozvrh není další vrstva šifrování zprávy.

9. SubBytes a ShiftRows krok za krokem

VAZBA NA PREZENTACI: SNÍMKY 3 A 8

SubBytes: pevný nelineární S-box

S-box má 256 položek a každý bajt nahradí právě jedním jiným bajtem. Je veřejný a pro všechny klíče AES stejný. Není to tajná proměnná kódová kniha.

Jeho konstrukce kombinuje multiplikativní inverzi v $GF(2^8)$ s pevnou afinní transformací bitů. Pro nulový vstup se v mezikroku použije nula. Nelinearita pochází z inverze; zbytek je přesně zvolený lineární převod s přičtenou konstantou pomocí XOR. Tak vznikne vratná, ale nelineární bajtová substituce.

Příklad čtení tabulky. Vstup 0x53 hledáme v řádku 5 a sloupci 3 tabulky S-boxu. Výsledek je **0xed**. Další ověřitelné hodnoty jsou S(0x00) = 0x63 a S(0xff) = 0x16. Opačný S-box vrátí například 0xed zpět na 0x53. [1]

V každé pozici se provede totéž pravidlo. SubBytes samo nemíchá sousední bajty: změna jednoho vstupního bajtu ovlivní v této fázi jen jeho náhradu.

ShiftRows: různé posuny řádků

Řádky 0, 1, 2 a 3 se cyklicky posunou vlevo o 0, 1, 2 a 3 bajty. Posun je v jednotkách bajtů, nikoli jednotlivých bitů. Pro názorně očíslovaný stav:

Řádek	Před posunem	Po posunu vlevo
0	00 04 08 0c	00 04 08 0c
1	01 05 09 0d	05 09 0d 01
2	02 06 0a 0e	0a 0e 02 06
3	03 07 0b 0f	0f 03 07 0b

Čísla v této tabulce pouze označují bajty; nejde o výsledek předchozího SubBytes.

Proč oba kroky potřebujeme

ShiftRows nemění hodnotu žádného bajtu. Roznáší ale původně společné sloupce mezi různé nové sloupce. Následující MixColumns tak míchá bajty s různou historií. Bez tohoto přesouvání by čtyři sloupce zůstávaly mnohem více oddělené.

> **Ověřte si intuici:** po samotném SubBytes je změna stále místní. Po ShiftRows je na jiné pozici. Po MixColumns se může projevit ve všech čtyřech bajtech daného sloupce. Další rundy šíří její vliv dále.

10. MixColumns: vypočítejte jeden sloupec

VAZBA NA PREZENTACI: SNÍMEK 8

MixColumns převede každý sloupec čtyř bajtů a0, a1, a2, a3 na nový sloupec b0, b1, b2, b3. Koeficienty násobení jsou pevné a veškerá aritmetika probíhá v $GF(2^8)$, nikoli nad běžnými celými čísly.

Výstup	Koef. a0	Koef. a1	Koef. a2	Koef. a3
b0	02	03	01	01
b1	01	02	03	01
b2	01	01	02	03
b3	03	01	01	02

Každý řádek tabulky říká, čím vynásobit příslušný vstupní bajt; čtyři výsledky spojíme XORem. Například b0 = (02·a0) ⊕ (03·a1) ⊕ a2 ⊕ a3.

Výpočet na konkrétních bajtech

Zvolme sloupec **d4 bf 5d 30**. Po MixColumns vznikne **04 66 81 e5**. První bajt dopočítáme samostatně:

```
02 · d4 = b3
03 · bf = (02 · bf) ⊕ bf = 65 ⊕ bf = da
b0 = b3 ⊕ da ⊕ 5d ⊕ 30 = 04
```

Pro d4 byl nejvyšší bit 1, proto po posunu dostaneme a8 a provedeme a8 ⊕ 1b = b3. Pro bf vznikne po posunu 7e a po redukci 7e ⊕ 1b = 65. Tato kontrola ukazuje, proč by běžné násobení dalo chybný výsledek.

Zbylé bajty obdobně vycházejí:

```
b1 = d4 ⊕ 65 ⊕ e7 ⊕ 30 = 66
b2 = d4 ⊕ bf ⊕ ba ⊕ 50 = 81
b3 = 67 ⊕ bf ⊕ 5d ⊕ 60 = e5
```

Hodnoty e7, ba, 50 a 67 jsou příslušné násobky v tělese. Například 03·5d = ba ⊕ 5d = e7 a 03·d4 = b3 ⊕ d4 = 67.

Co lineární transformace zajišťuje

Každý výstupní bajt závisí na všech čtyřech vstupních. Matice je invertibilní, takže existuje InvMixColumns. Linearita není sama o sobě problém: tento krok má rozptylovat vliv vstupu a skládá se s nelineární substitucí a klíčováním.

> **Zapamatujte si:** MixColumns není náhodné míchání ani změna délky. Jde o přesně daný vratný převod čtyř bajtů na čtyři bajty. Ve finální rundě AES se nepoužívá.

11. Podklíče, AddRoundKey a dešifrování

VAZBA NA PREZENTACI: SNÍMEK 8

Jak se základní klíč rozvine

KeyExpansion vytváří klíčový rozvrh. U AES-128 se 16 vstupních bajtů rozdělí na čtyři 32bitová slova w0 až w3 a odvodí se celkem 44 slov: jedenáct 16bajtových podklíčů. Pro AES-192 jde o 52 slov a pro AES-256 o 60 slov. Podklíče nejsou nezávislá náhodná tajemství; veškerou jejich informaci určuje základní klíč.

U AES-128 se každé čtvrté nové slovo vytváří pomocí cyklického posunu bajtů **RotWord**, dosazení S-boxu **SubWord** a rundové konstanty **Rcon**. Další slova využijí XOR s předchozím slovem a se slovem o čtyři pozice dříve. Varianty s delším klíčem mají upravená pravidla.

Pro základní klíč `000102030405060708090a0b0c0d0e0f` vychází první nové slovo:

```
w3               = 0c 0d 0e 0f
RotWord(w3)      = 0d 0e 0f 0c
SubWord(...)     = d7 ab 76 fe
XOR Rcon[1]      = d6 ab 76 fe
w4 = w0 ⊕ (...) = d6 aa 74 fd
```

Rcon[1] je slovo `01 00 00 00`. První odvozený podklíč pro rundu 1 je `d6aa74fdd2af72fadaa678f1d6ab76fe`.

AddRoundKey

Každý bajt stavu se XORuje s odpovídajícím bajtem podklíče. Výpočet je jednoduchý, ale přimíchává tajemství do celé konstrukce. Operace je vlastní inverzí: (s ⊕ k) ⊕ k = s. Z názvu „Add“ se nesmí odvozovat obyčejné sčítání s přenosem.

Jak se šifrování obrací

Přímý inverzní algoritmus nejprve přidá poslední podklíč. Poté opakuje InvShiftRows, InvSubBytes, AddRoundKey a InvMixColumns s podklíči v opačném pořadí. Poslední inverzní kolo končí původním podklíčem a opět neobsahuje InvMixColumns. [1]

Na rozdíl od Feistelovy intuice potřebujeme u přímého inverzního AES také inverzní transformace. Standard popisuje i ekvivalentní uspořádání dešifrování s upraveným rozvrhem; to však nemění šifrový výsledek ani základní klíč.

> **Prostor klíčů se expanzí nezvětší.** Z 128bitového klíče nevzniká 1 408 bitů nezávislého tajemství, i když jedenáct podklíčů dohromady zabere tolik místa.

12. Ověřovací příklad celého AES-128

PROPOJENÍ STAVU, RUND A KLÍČOVÉHO ROZVRHU

Použijeme známý ověřovací vstup AES. Všechny řetězce jsou hexadecimální a představují bajty v původním pořadí, tedy stav čtený **po sloupcích**. Jeden 32znakový řetězec je 16 bajtů, nikoli 32 bajtů.

```
Klíč K:  000102030405060708090a0b0c0d0e0f
Vstup P: 00112233445566778899aabbccddeeff
```

Počáteční krok

První podklíč je u AES-128 původní klíč. Po AddRoundKey proto vznikne `00102030405060708090a0b0c0d0e0f0`. První bajt je 00 ⊕ 00 = 00, druhý 11 ⊕ 01 = 10 a poslední ff ⊕ 0f = f0.

První runda

Fáze	Stav jako posloupnost 16 bajtů
Počáteční AddRoundKey	00102030405060708090a0b0c0d0e0f0
SubBytes	63cab7040953d051cd60e0e7ba70e18c
ShiftRows	6353e08c0960e104cd70b751bacad0e7
MixColumns	5f72641557f5bc92f7be3b291db9f91a
AddRoundKey pro rundu 1	89d810e8855ace682d1843d8cb128fe4

Například první sloupec po ShiftRows je `63 53 e0 8c`. MixColumns ho převede na `5f 72 64 15`. Potom se XORuje s prvními čtyřmi bajty podklíče `d6 aa 74 fd`; vznikne `89 d8 10 e8`.

Výsledek po deseti rundách

```
C = 69c4e0d86a7b0430d8cdb78070b4c55a
```

Příklad dovoluje oddělit chyby. Nesprávný výsledek hned po prvním XORu ukazuje na vstup nebo podklíč. Shoda po SubBytes a neshoda po ShiftRows často znamená chybné plnění stavu. Neshoda po MixColumns může ukazovat na záměnu aritmetiky tělesa za běžná čísla.

Co příklad neověřuje: jediný známý vektor není důkazem bezpečnosti ani úplnosti implementace. Neprověřuje všechny vstupy, postranní kanály ani bezpečnou správu klíčů. V těchto skriptech slouží ke kontrole výkladu a výpočtů.

> **Učení po částech:** není nutné ručně spočítat všech deset rund. Důležité je umět objasnit každou mezifázi první rundy a vědět, která operace mění hodnoty a která pouze pozice bajtů.

13. Režimy použití a doplnění zprávy

VAZBA NA PREZENTACI: SNÍMKY 9-10

Režim určuje, jak opakovaně použít základní transformaci E_K na celou zprávu. Určuje souvislosti mezi bloky a pomocné vstupy. Stejný AES může být součástí režimu zachovávajícího opakované vzory i součástí autentizovaného šifrování.

Pro režimy dále značíme původní bloky P1, P2, … a výsledné bloky C1, C2, …. Každý plný blok má u AES 16 bajtů. **IV** je inicializační hodnota, **nonce** vstup, který se podle konstrukce nesmí opakovat. Název sám neurčuje, zda hodnota musí být náhodná či nepředvídatelná.

Proč někdy potřebujeme padding

Režimy ECB a základní CBC potřebují vstup složený z celých bloků. Když zpráva nevyplní poslední blok, doplníme ji jednoznačně odstranitelnou výplní. Jednou známou variantou je **PKCS#7**: doplní q bajtů a každý z nich má číselnou hodnotu q.

Původní délka při AES	Doplňované bajty v hex	Celková délka
14 bajtů	02 02	16 bajtů
15 bajtů	01	16 bajtů
16 bajtů	Šestnáct bajtů 10	32 bajtů

Přidání celého bloku u zarovnané zprávy odstraňuje nejednoznačnost: bez něj by nebylo jasné, zda poslední bajt 01 pochází z dat, nebo je výplní. Způsob doplnění musí být součástí specifikace komunikace.

CTR, OFB a GCM umějí zpracovat i neúplný poslední blok pomocí odpovídající části proudu; nepotřebují stejný padding zprávy jako CBC. Vnitřní zarovnávání autentizační funkce GCM je jiná věc a nesmí se zaměňovat s přidáváním bajtů k původní zprávě.

Blok není sektor

AES blok má 16 bajtů. Disková datová jednotka může mít například 512 nebo 4096 bajtů, takže obsahuje mnoho AES bloků. Režim XTS definuje práci s takovou jednotkou; její délka není novou velikostí bloku AES.

> **Volba režimu mění záruky, nikoli parametry AES.** Zvolená implementace nemůže udělat z AES-256 šifru se 4096bajtovým blokem. Může jen seskupit a zpracovat mnoho pevných bloků.

14. ECB: proč mohou zůstat vidět vzory

VAZBA NA PREZENTACI: SNÍMKY 10-11

Electronic Codebook (ECB) šifruje každý blok nezávisle stejným klíčem. Pro každý index i platí:

```
C_i = E_K(P_i)
P_i = D_K(C_i)
```

Protože je základní šifra pro pevný klíč deterministická, platí P_i = P_j právě tehdy, když C_i = C_j. Útočník nemusí znát obsah bloku, aby rozpoznal, že se některé hodnoty opakují.

Ukázka na symbolických blocích

Písmena níže zastupují celé 16bajtové bloky, nikoli jednotlivé znaky. X je šifrování bloku A, Y bloku B a Z bloku C pro jeden konkrétní klíč.

Pozice	1	2	3	4	5
Otevřené bloky	A	B	A	C	A
Výsledek ECB	X	Y	X	Z	X

Z výsledku je vidět, které bloky byly stejné. To platí i pro AES-256: větší klíč pomáhá proti hledání klíče, ale tuto vlastnost režimu neodstraní.

Co ukazuje obrázek tučňáka

U vhodně připraveného nezkomprimovaného obrázku se opakují bloky stejných barev či vzorů. ECB jejich rovnost zachová, a proto mohou zůstat patrné obrysy. Původní barvy neznáme, ale struktura obrazu uniká.

Neznamená to, že ECB zachová rozpoznatelný obrys každé fotografie v každém formátu. Komprimované obrázky mohou mít jiné opakování bajtů a jejich struktura není totožná s prostorovým rozmístěním pixelů. Příklad ilustruje únik rovnosti bloků, nikoli univerzální vizuální efekt.

Výkon není bezpečnostní cíl

ECB lze snadno paralelizovat při šifrování i dešifrování a umožňuje samostatný přístup k blokům. Tyto výhody ale neodstraní únik vzorů ani nezajistí autenticitu. Bloky lze v některých formátech přesouvat či nahrazovat, aniž by šifra sama oznámila chybu.

> **Odlišení algoritmu a režimu:** rozpoznatelný obrázek v ECB není praktické prolomení AES. AES správně a nezávisle zašifroval každý blok; problémem je právě jejich nezávislé použití při stejném klíči.

15. CBC: řetězení šifrových bloků

VAZBA NA PREZENTACI: SNÍMEK 12

Cipher Block Chaining (CBC) nejprve XORuje původní blok s předchozím šifrovým blokem. Teprve výsledek pošle do blokové šifry. Pro první krok hraje roli předchozího bloku IV.

```
C_0 = IV
C_i = E_K(P_i ⊕ C_(i-1))
P_i = D_K(C_i) ⊕ C_(i-1)
```

Dešifrovací vzorec plyne přímo z D_K(E_K(x)) = x. Po aplikaci D dostaneme P_i ⊕ C_(i-1); druhý XOR s C_(i-1) obnoví P_i.

Co řetězení mění

Dva stejné bloky P_i a P_j zpravidla vstupují do AES po XORu s různými předchozími bloky. Nevytvářejí tedy automaticky stejný šifrový blok jako v ECB. Identické celé zprávy se stejným klíčem a stejným IV však vytvoří stejné výsledky; IV musí být správně zvoleno.

Pro CBC má být IV pro dané šifrování **nepředvídatelné**. Samotná záruka „nikdy se neopakuje“ nestačí k obecnému naplnění tohoto požadavku. Přímé použití veřejného čítače jako IV tedy není totéž co správné CBC IV. Hodnota nemusí zůstat tajná a běžně se posílá se zprávou. Její integrita musí být také chráněna. [5]

Jednoduchá algebraická kontrola

Pokud pro konkrétní blok vyjde D_K(C_i) = `1010` a předchozí blok je `0110`, dešifrovaná čtyřbitová hračka je `1010 ⊕ 0110 = 1100`. U skutečného AES probíhá stejný XOR na 128 bitech; zvolená čtyřbitová čísla pouze zjednodušují sledování vzorce.

Paralelizace

Při šifrování nelze spočítat C2 bez C1, takže jedna zpráva vytváří sériový řetězec. Při dešifrování už všechny C_i známe: D_K(C1), D_K(C2) a další výpočty mohou běžet paralelně a pak se provede příslušný XOR. Různé nezávislé zprávy lze při vhodném zpracování šifrovat souběžně.

> **CBC samo neověřuje původ ani neporušenost.** Potřebuje správný autentizační protokol. Pro nový systém se proto obvykle používá standardní hotové AEAD rozhraní místo vlastního skládání CBC a MAC.

16. Chyby CBC a rozdíl proti CBC-MAC

UPŘESNĚNÍ A DOPLNĚNÍ SNÍMKU 12

Šifrování změněné zprávy vs. změna přenosu

Pokud při šifrování změníme původní P_i, změní se C_i a jeho prostřednictvím zpravidla i všechny další šifrové bloky. To je jiná situace než poškození již hotového C_i během přenosu.

Při dešifrování poškozeného C_i bude P_i obecně rozsáhle změněn. V **P_(i+1)** se navíc převrátí přesně ty bity, které útočník změnil v C_i. Pozdější bloky už takovou změnou nejsou ovlivněny, pokud se nezmění jejich pořadí ani hranice. Změna samotného IV převrátí odpovídající bity jen v P1.

Tyto vlastnosti dovolují cílené manipulace, ne jen náhodné poškození. Pokud se z reakcí na chybnou výplň dozvídáme další informace, může vzniknout **padding oracle**. Odlišování „špatné výplně“ a jiných chyb není autentizační ochrana.

CBC-MAC je jiný úkol

CBC-MAC používá řetězení k vytvoření ověřovacího kódu: typicky se začne nulovým stavem a jako tag se vezme poslední hodnota řetězce. Původní zpráva zůstává dostupná; cílem je autenticita a integrita, nikoli utajení.

Jednoduché CBC-MAC má bezpečnostní podmínky, mimo jiné omezení použití na vhodně vymezené délky zpráv. Nelze bez dalších úprav přijímat všechny délky jen proto, že vzorec umí řetězit libovolný počet bloků.

Proč je proměnná délka problém

Pro jeden plný blok M a nulový počátek je tag t = E_K(M). Kdo zná M i jeho tag, může sestavit dvoublokovou zprávu M || (M ⊕ t), kde || znamená spojení:

```
První stav = E_K(M) = t
Druhý stav = E_K((M ⊕ t) ⊕ t) = E_K(M) = t
```

Stejný tag t tedy platí i pro novou zprávu, pokud naivní ověřovač bez ochrany délky přijímá jeden i dva bloky. To je konkrétní padělek, ne prolomení AES.

Standardní **CMAC** upravuje mimo jiné zpracování posledního bloku a umožňuje bezpečně řešit různě dlouhé zprávy podle své specifikace. [6] Ani CMAC není šifrování a nepřidává důvěrnost.

> **Dvě časté záměny:** šíření chyby není ověření integrity; a CBC šifrování není totéž co CBC-MAC, přestože sdílejí část mechanismu.

17. CFB, OFB a PCBC v jednom přehledu

ROZVEDENÍ DIAGRAMŮ ZE SNÍMKU 10

CFB: zpětná vazba šifrového textu

Pro jednoduchost popíšeme **plnoblokové CFB**, tedy CFB-128 u AES. Existují i varianty pracující s kratšími segmenty. Začneme C0 = IV a počítáme:

```
S_i = E_K(C_(i-1))
C_i = P_i ⊕ S_i
P_i = C_i ⊕ E_K(C_(i-1))
```

I pro dešifrování používáme E_K, nikoli D_K. Výstup dopředné transformace zde slouží jako maska. Šifrování má závislost na předchozím výsledku. Při dešifrování plnoblokové varianty lze příslušné výpočty E_K předchozích známých šifrových bloků provádět souběžně.

Změna bitu C_i cíleně změní stejný bit P_i a zpravidla rozsáhle poškodí P_(i+1); další bloky se obnoví. Zde nejde o stejný vzorec šíření jako u CBC. CFB vyžaduje nepředvídatelné IV. [5]

OFB: zpětná vazba vlastního výstupu

Output Feedback (OFB) vytváří proud nezávisle na obsahu zprávy: S0 = IV, S_i = E_K(S_(i-1)) a C_i = P_i ⊕ S_i. Stejným XORem dešifrujeme. Proud můžeme připravit před příchodem dat, ale jeho bloky vznikají postupně, protože každý potřebuje předchozí S_i.

Pro jeden klíč se IV nesmí opakovat, jinak se opakuje celý proud. OFB nevyžaduje nepředvídatelnost IV stejným způsobem jako CBC; zásadní je předepsaná jedinečnost. Převrácení bitu v C_i převrátí jen odpovídající bit zprávy. To pomáhá tolerovat náhodné bitové chyby, ale dovoluje i manipulaci. [5]

PCBC: propagující řetězení

Propagating CBC zahrnuje do dalšího kroku předchozí původní i šifrový blok. Zavedeme stav V1 = IV, C_i = E_K(P_i ⊕ V_i) a V_(i+1) = P_i ⊕ C_i. Inverze je P_i = D_K(C_i) ⊕ V_i.

Cílem je rozsáhlejší šíření změn. Ani to však není kryptografická autentizace. PCBC nepatří mezi pět režimů standardu SP 800-38A a není důvodem upřednostnit jej před hotovým moderním AEAD schématem.

> **Hlavní rozdíl:** CFB vrací do vstupu šifrový text, OFB svůj interní výstup a PCBC kombinaci původního i šifrového textu. Všechny názvy „feedback“ proto neoznačují stejný tok dat.

18. CTR: ze šifry vznikne klíčový proud

VAZBA NA PREZENTACI: SNÍMEK 10

Counter mode (CTR) šifruje posloupnost různých vstupů čítače. Výsledné bloky kombinuje XORem s daty:

```
S_i = E_K(T_i)
C_i = P_i ⊕ S_i
P_i = C_i ⊕ S_i
```

T_i je celý vstupní blok čítače, například vhodné spojení nonce a čísla bloku. Přesný formát určuje protokol. Nemusí být tajný; pro daný klíč se však **nesmí znovu použít tentýž T_i**, ani uvnitř jedné zprávy, ani napříč zprávami. [5]

Proč je CTR rychlé a praktické

Vstupy T_i známe nezávisle na zprávě a předchozích výsledcích. Všechny výpočty E_K(T_i) lze při dostatku prostředků paralelizovat pro šifrování i dešifrování. Pro blok na vzdálené pozici stačí získat příslušný vstup čítače.

Kratší poslední blok se XORuje jen s potřebným počtem bajtů proudu. Například 20bajtová zpráva využije 16 bajtů prvního proudového bloku a čtyři bajty druhého. Padding jako u CBC není nutný. Nesmíme však překročit povolený počet bloků a nechat čítač přetéct do už použité hodnoty.

Opakovaný proud znovu odhalí vztah zpráv

Pokud je pro dva původní bloky použit stejný proud S, platí $C \oplus C' = P \oplus P'$. Pro jednoduché čtyřbitové hodnoty:

```
S = 1010; P = 1100; P' = 0011
C = 0110; C' = 1001
C ⊕ C' = 1111 = P ⊕ P'
```

Pokud navíc známe P, vypočteme $S = C \oplus P$ a přečteme P'. Zprávy nevytvářejí správné OTP: dlouhý proud se deterministicky odvozuje z krátkého klíče a jeho bezpečnost je výpočetní.

CTR také nezajišťuje integritu

Změna C na $C \oplus \Delta$ vytvoří po dešifrování $P \oplus \Delta$. Útočník může vyvolat přesné bitové změny i bez klíče, pokud mu formát zprávy dovolí zacílení. Náhodně vypadající šifrový text tuto vlastnost neodstraňuje.

> **Spojení s GCM:** GCM využívá čítačový princip pro důvěrnost, ale přidává zvlášť definovanou autentizační vrstvu. Samotný CTR tuto vrstvu nemá.

19. IV, nonce a srovnání režimů

VAZBA NA PREZENTACI: SNÍMKY 9-14

Pomocný vstup zpravidla nemusí být tajný. Jeho bezpečnostní podmínka ale závisí na režimu. **Jedinečné** znamená neopakující se, **nepředvídatelné** znamená, že jej protivník nemůže předem odhadnout s využitelnou výhodou. Jedna vlastnost automaticky nenahrazuje druhou.

Režim	Podmínka pomocného vstupu při stejném klíči	Padding zprávy	Autentizace
ECB	Žádné IV nemá	Pro základní variantu ano	Ne
CBC	Nepředvídatelné IV; chránit i jeho integritu	Pro základní variantu ano	Ne
CFB-128	Nepředvídatelné IV	Nevyžaduje PKCS#7; záleží na segmentaci rozhraní	Ne
OFB	Jedinečné IV dle konstrukce	Ne	Ne
CTR	Žádný opakovaný vstup čítače	Ne	Ne
GCM	Neopakovat nonce/IV; dodržet limity konstrukce	Ne	Ano
XTS	Tweak odpovídá datové jednotce a pozici	Nepoužívá PKCS#7	Ne

Proč nestačí „vygenerujeme něco náhodného“

Náhodné hodnoty se mohou střetnout. U rovnoměrných nbitových hodnot roste pravděpodobnost alespoň jedné kolize přibližně jako $q(q - 1)/2^{n+1}$ pro q generování, pokud je tato pravděpodobnost malá. Proto se musí hlídat délka nonce i počet zpráv na klíč. Čítač se zase může opakovat po restartu nebo chybné obnově stavu.

Závislosti při zpracování jedné zprávy

Režim	Paralelní šifrování bloků	Paralelní dešifrování bloků
ECB	Ano	Ano
CBC	Ne, závisí na předchozím výsledku	Ano
CFB-128	Ne, závisí na předchozím výsledku	Ano, z již známých šifrových bloků
OFB	Proud vzniká sériově; lze ho předpočítat	Stejný sériový proud
CTR	Ano	Ano

Přehled posuzuje matematické závislosti, nikoli zaručenou rychlost konkrétního zařízení. Různé zprávy mohou být nezávislé i tam, kde je řetězení uvnitř jedné zprávy sériové.

> **XTS je zvláštní případ:** při přepisu stejné diskové pozice se její identifikátor přirozeně opakuje. Pravidlo nonce pro GCM na XTS nepřenášíme mechanicky; cíle a záruky konstrukcí se liší.

20. GCM: čítač a autentizace společně

VAZBA NA PREZENTACI: SNÍMEK 13

Galois/Counter Mode (GCM) kombinuje čítačové šifrování s autentizační funkcí. Vytváří šifrový text C a tag T. Chrání původní obsah i přidružená data A, která jsou ověřována, ale nešifrují se. Jde o režim typu AEAD. [7]

Dvě propojené větve

V šifrovací větvi se AES aplikuje na odvozené bloky čítače a vzniklý proud se XORuje s původními daty. V autentizační větvi funkce **GHASH** zpracuje přidružená data, šifrový text a údaje o jejich délkách. Výsledek se zkombinuje s další klíčově odvozenou hodnotou a vytvoří tag.

GHASH používá násobení v **$GF(2^{128})$**, nikoli v bajtovém $GF(2^8)$ uvnitř AES. To jsou dvě různá tělesa v různých částech konstrukce. Násobicí klíč GHASH se odvozuje z AES, typicky $H = E_K(0^{128})$.

Samo násobení v tělese není tajným šifrováním zprávy. GHASH je součást přesně vymezeného ověřovacího schématu a bez ostatních prvků se nesmí vydávat za bezpečný univerzální MAC.

Co posíláme a co ověřujeme

Modelový záznam nese nonce, přidružená data nebo jejich kontext, šifrový text a tag. Příjemce musí mít stejné A a nonce, aby ověření vyšlo. Změna ověřované hlavičky způsobí selhání, i když sama hlavička zůstává veřejná.

V běžném nastavení se používá **96bitové nonce** a často **128bitový tag**. Nejde o délku bloku AES ani délku klíče. Parametry a limity je nutné brát ze specifikace protokolu; historická podpora kratších tagů není důvodem je svévolně zvolit. NIST připravuje revizi pravidel GCM. [7, 8]

Proč je opakování nonce závažné

Stejný klíč a nonce znovu použijí proud pro důvěrnost a poruší také předpoklady autentizace. Následky tedy nejsou omezeny jen na vztah mezi dvěma zprávami; mohou umožnit i padělání. Ani dlouhý tag neopraví opakované nonce.

> **Bezpečnostní pravidlo:** aplikace nesmí používat neověřený otevřený text jako důvěryhodný výsledek. AEAD dešifrování má poskytnout ověřenou zprávu, nebo neúspěch. Data uvolněná knihovnou před finálním ověřením zůstávají do té doby nedůvěryhodná.

21. AEAD v protokolu a v aplikaci

VAZBA NA PREZENTACI: SNÍMEK 14

Authenticated Encryption with Associated Data je standardní rozhraní, které spojuje důvěrnost s autentizací. Zjednodušeně:

```
Encrypt(K, N, P, A) → (C, T)
Decrypt(K, N, C, A, T) → P nebo CHYBA
```

K je klíč, N nonce, P původní data, A přidružená data, C šifrový text a T tag. Úspěšné dešifrování není pouze převod bajtů; zahrnuje ověření podle konstrukce.

Co dát do přidružených dat

Představte si zprávu s veřejnou hlavičkou „typ=platba, verze=2, pořadí=17“ a tajným obsahem. Hlavičku lze zahrnout do A. Příjemce ji přečte bez dešifrování, ale útočník ji nemůže bez detekce změnit společně s chráněným záznamem.

AEAD samo nepřikazuje, co všechno do A zahrnout. Pokud aplikace zapomene ověřit například identifikátor záznamu, může protokol připustit záměnu kontextu, i když jednotlivé tagy zůstávají kryptograficky platné.

Běžné konstrukce

AES-GCM bývá velmi výkonné při hardwarové podpoře AES a násobení potřebného pro GHASH. **ChaCha20-Poly1305** kombinuje proudovou šifru a autentizaci a je vhodně navrženo také pro efektivní software. ChaCha20 není bloková šifra; zmiňujeme ji pro srovnání celých AEAD konstrukcí. [9]

TLS 1.3 používá pro ochranu záznamů AEAD; jeho specifikace zahrnuje AES-GCM, ChaCha20-Poly1305 a varianty AES-CCM. WireGuard používá ChaCha20-Poly1305 pro symetrickou ochranu svého protokolu. Neznamená to, že tyto protokoly žádné další kryptografické prostředky nepotřebují. [10, 11]

Co AEAD neřeší automaticky

Opakované doručení stejné platné zprávy může mít stále platný tag. **Ochranu proti replay** musí zajistit protokol, například kontrolou pořadových čísel a stavu. AEAD také neschová velikost ani okamžik přenosu a neochrání klíč uložený v napadeném procesu.

> **Správné použití:** standardní knihovna, přípustné parametry, správa jedinečných nonce a vyhodnocení výsledku ověření. Tvrzení, že algoritmus „běží v konstantním čase“, musí platit o konkrétní implementaci a prostředí, nikoli jen o jeho názvu.

22. XTS: proč disk není síťová zpráva

ROZVEDENÍ POZNÁMKY NA SNÍMKU 9

Diskové šifrování potřebuje nezávisle číst a přepisovat malé části úložiště, často bez možnosti zvětšit každý sektor o nonce a tag. **XTS-AES** je režim určený k ochraně důvěrnosti dat na úložišti v takovém prostředí. **Neposkytuje autentizaci dat ani jejich zdroje.** [12]

Blok, datová jednotka a tweak

AES stále zpracovává 16bajtové bloky. Datová jednotka, například 512bajtový sektor, obsahuje 32 takových bloků. Jednotka 4096 bajtů jich obsahuje 256. Každý blok se šifruje s vlivem pozice, aby stejné původní bloky na různých místech automaticky nedávaly stejné výsledky.

Ve zjednodušeném popisu plných bloků má XTS tvar:

```
T_j = odvozený tweak pro jednotku a pozici j
C_j = E_K1(P_j ⊕ T_j) ⊕ T_j
```

Základ pro tweak se vytváří pomocí druhého klíče K2 z identifikátoru datové jednotky; pro další pozice se upravuje podle definovaného pravidla. Číslo sektoru je tedy možné **vstupní označení jednotky**, nikoli obecně celý hotový IV ve smyslu CBC nebo GCM.

Dva klíče a délka

XTS používá dvě klíčové části. AES-128-XTS potřebuje celkem 256 klíčových bitů, AES-256-XTS celkem 512. To neznamená 512bitový blok AES ani automaticky 512bitovou odolnost proti hledání klíče. Je nutné číst označení konkrétního rozhraní.

Když má přípustná datová jednotka neúplný poslední blok, používá XTS předepsané **ciphertext stealing**, tedy zvláštní zpracování závěrečných částí bez zvětšení celkové délky. Nejde o PKCS#7 a jednotka musí mít alespoň jeden plný blok. Přesné limity jsou součást specifikace.

Hranice ochrany

Při přepisu stejné pozice se identifikátor přirozeně opakuje. XTS neslibuje stejné vlastnosti jako nové náhodnostní šifrování každé síťové zprávy. Útočník může například vrátit dříve uložený šifrový obsah téže pozice a samotné XTS neposkytuje tag, který by takový replay odhalil.

> **Praktický rozdíl:** šifrování disku chrání obsah zejména při získání úložiště bez klíče. Ověření neporušenosti, aktuálnosti a konzistence dat vyžaduje další vrstvu, pokud je to cílem systému.

23. Co přesně znamená útok na AES

VAZBA NA PREZENTACI: SNÍMEK 15

Hrubá síla

Pro náhodný kbitový klíč existuje 2^k možností. Bez lepšího útoku potřebuje systematické hledání v průměru přibližně polovinu prostoru, tedy 2^{k-1} zkoušek; řád složitosti bývá zapisován $O(2^k)$. Pro AES-256 je proto 2^{256} označení velikosti prostoru, nikoli přesný počet operací každého úspěšného útoku.

Útočník navíc potřebuje způsob rozpoznání správného klíče. Jeden známý blok, struktura zprávy nebo platný tag poskytují různé ověřovací informace. Doba jedné zkoušky, paměť, potřebná data a možnost paralelizace jsou součást celkových nákladů.

Biclique útok

Práce Bogdanova, Khovratoviche a Rechbergera z roku 2011 uvedená v prezentaci popsala získání klíče plného AES-128 s časovou složitostí přibližně **2^(126,1)**, tedy s exponentem 126,1. V technických zápisech s desetinnou tečkou jde o 2^(126.1). Jde o odborný výsledek lepší než jednoduchá referenční hrubá síla v daném modelu, nikoli o praktické přečtení AES komunikace. [13]

Poměr 2^{128} / 2^(126.1) = 2^(1,9) je přibližně 3,73. Číslo v exponentu se zmenšilo nepatrně a zbývající práce je stále nesmírná. Navíc musíme porovnávat stejné jednotky práce a podmínky, nikoli jen dva izolované exponenty.

Co při čtení výsledků kontrolovat

- Jde o plný algoritmus, nebo jen menší počet rund?
- Útok předpokládá jeden klíč, nebo speciální vztahy mezi více klíči?
- Jaké množství dat, paměti a přístupu ke službě vyžaduje?
- Získá klíč, rozezná šifru od náhodné permutace, nebo jen poruší dílčí vlastnost?
- Je prakticky realizovatelný, nebo ukazuje teoretickou hranici návrhu?

Tento seznam vysvětluje, proč „existuje útok“ není bez dalšího rozboru totéž co „algoritmus přestal chránit běžná data“. Současně nelze útok ignorovat jen proto, že je zatím drahý; může odkrýt důležitou konstrukční vlastnost.

> **Pozor na datování:** citovaný výsledek je konkrétní práce z roku 2011. Neprezentujeme jej jako automaticky nejnovější či nejlepší výsledek všech bezpečnostních modelů.

24. Implementace, kvantové útoky a člověk

VAZBA NA PREZENTACI: SNÍMKY 2 A 15

Postranní kanály a hardwarová podpora

Tabulková softwarová implementace může podle tajných hodnot přistupovat na různá místa paměti. Z časování a chování cache pak může unikat informace, i když je matematický výsledek AES správný. **Konstantní čas** označuje relevantní nezávislost chování výpočtu na tajných hodnotách, nikoli doslova stejné měření stopkami v každé situaci.

Specializované instrukce AES mohou odstranit určité rizikové tabulkové přístupy a výrazně zrychlit výpočet. Neznamená to eliminaci všech postranních kanálů. Zůstává otázka správy klíčů, dalších částí protokolu a fyzických projevů, například spotřeby nebo elektromagnetického vyzařování. Také úmyslně vyvolané chyby výpočtu tvoří jinou skupinu útoků.

Stejně tak vhodná konstrukce ChaCha20 usnadňuje bezpečnou softwarovou implementaci, ale slogan „je konstantního času“ nenahrazuje kontrolu konkrétní knihovny a platformy.

Groverův algoritmus

Ideální kvantové hledání může snížit počet dotazů do prohledávání prostoru velikosti 2^k přibližně na řád **2^(k/2)**. Pro klíč 128 bitů je to řád 2^{64}, pro 256 bitů 2^{128}. To není Shorův algoritmus pro problémy typu faktorizace a neznamená okamžité praktické prolomení AES.

Jedna kvantová zkouška není obyčejné vyzkoušení klíče na běžném procesoru. Potřebujeme obrovský realizovatelný kvantový výpočet, korekci chyb a složité obvody; paralelizace má jiné náklady. NIST proto hodnotí dopad s ohledem na skutečné zdroje, nikoli jen mechanickým půlením délky klíče. [14]

Prohlášení „AES je odolné vůči kvantovým počítačům“ je tedy třeba číst jako tvrzení o parametrech a známých nákladech útoků, ne jako matematickou imunitu proti jakémukoli kvantovému zrychlení.

Proč prezentace začíná komiksem

Komiks na snímku 2 upozorňuje na obejití šifry získáním hesla od člověka. Malware podobně přečte dokument před zašifrováním či po dešifrování. Silnější klíč neodstraní útok na místo, kde obsah už existuje otevřeně.

> **Celý systém:** dobře navržená šifra, správný režim, kvalitní klíč, bezpečná implementace a ochrana koncového zařízení řeší různé vrstvy. Selhání jedné nelze automaticky napravit prodloužením klíče v jiné.

25. Serpent, Twofish a další finalisté

VAZBA NA PREZENTACI: SNÍMKY 16-18

Do finále soutěže AES postoupilo pět algoritmů. Rijndael se stal standardem, ale Serpent, Twofish, MARS ani RC6 tím automaticky nebyly prohlášeny za prolomené. Soutěž hledala vhodný standard v několika praktických kritériích, ne vítěze jediné matematické disciplíny. [2]

Přijatí kandidáti prvního kola: CAST-256, CRYPTON, DEAL, DFC, E2, FROG, HPC, LOKI97, MAGENTA, MARS, RC6, Rijndael, SAFER+, Serpent a Twofish. Tento seznam popisuje účast, nikoli jejich současnou vhodnost.

Serpent

Autoři **Ross Anderson, Eli Biham a Lars Knudsen** navrhli 128bitovou blokovou šifru s klíči 128, 192 a 256 bitů a **32 rundami**. Konstrukce je substitučně-permutační, se střídáním klíčových a transformačních vrstev. Návrh zdůrazňuje konzervativní bezpečnostní rezervu. [15]

Pracuje s osmi různými 4bitovými S-boxy. V jedné příslušné substituční vrstvě se zvolený S-box uplatní paralelně na 32 čtveřic bitů, což pokryje 128bitový stav. **Bitslicing** zpracovává tyto logické operace paralelně napříč bitovými rovinami strojových slov. Neznamená to nutnost spouštět 32 samostatných procesů.

Twofish

Autoři **Bruce Schneier, John Kelsey, Doug Whiting, David Wagner, Chris Hall a Niels Ferguson** navrhli 128bitový blok, klíče 128/192/256 bitů a **16 rund**. Jde o Feistelovskou konstrukci s dalšími operacemi. [16]

Výrazným rysem jsou **S-boxy závislé na klíči**, oproti pevné tabulce AES. Dále používá maticové míchání, rotace, součty modulo 2^{32} a klíčové vrstvy před hlavními rundami i po nich, označované jako **whitening**. XOR, rotace a běžný součet modulo 2^{32} jsou různé operace a nelze je zaměnit.

To, že Twofish a DES patří do příbuzné konstrukční rodiny, nepřenáší automaticky slabiny DES na Twofish. Rozhodují parametry, rundová funkce, rozvrh klíče a konkrétní analýza.

Proč neporovnávat jen počet rund

Serpent má 32 rund a AES-128 deset, ale jejich rundy provádějí jinou práci. Stejně tak klíčově závislý S-box není sám o sobě důkaz vyšší bezpečnosti. V praxi rozhoduje i dostupnost ověřených knihoven, hardwarová podpora a požadavky protokolu.

> **Ke zkoušce:** umět přiřadit AES a Serpent k substitučně-permutačním konstrukcím, Twofish k Feistelovské rodině a všem třem 128bitový blok. Není nutné učit se počty hlasů jako pořadí kryptografické bezpečnosti.

26. Camellia a evropský projekt NESSIE

VAZBA NA PREZENTACI: SNÍMKY 19-20

Camellia: další samostatný návrh

Camellia vznikla spoluprací NTT a Mitsubishi Electric. Má blok 128 bitů a klíče 128, 192 nebo 256 bitů. Používá Feistelovskou konstrukci s dalšími vnitřními vrstvami. Pro 128bitový klíč má 18 hlavních Feistelových rund, pro 192 a 256 bitů 24; kromě nich pracuje také s předepsanými klíčovými a logickými vrstvami. [17]

Přirovnání „japonský ekvivalent AES“ vystihuje podobné použití a parametry, nikoli shodnou konstrukci. Camellia není varianta Rijndaelu a při stejném klíči a zprávě nedává stejný výsledek. Název podle kamélie je historická zajímavost, ne bezpečnostní argument.

Specifikaci popisuje RFC 3713. Výběr do hodnoticích portfolií a standardizační procesy představují určitý odborný kontext; samotné zařazení ale nenahrazuje bezpečné použití v konkrétním systému.

Co bylo NESSIE

New European Schemes for Signatures, Integrity and Encryption byl evropský výzkumný a hodnoticí projekt v letech 2000-2003. Na rozdíl od soutěže AES se neomezoval na jedinou blokovou šifru: zabýval se širší sadou kryptografických prostředků pro šifrování, integritu a podpisy.

Prezentace uvádí mezi blokovými šiframi portfolia **MISTY1, Camellia, SHACAL-2 a AES**. Materiály k výběru zveřejňují i tvůrci Camellie; její zařazení potvrzuje rovněž RFC 3713. Jde o historické portfolio, nikoli automatické současné doporučení každé položky pro libovolný nový systém. [17, 18]

Jak číst podobná schválení

Je rozdíl mezi popisem algoritmu v dokumentu RFC, začleněním do standardu, doporučením hodnoticího projektu a skutečnou podporou v protokolu. Zároveň může algoritmus vhodný pro jednu aplikaci vyžadovat jiné podmínky než algoritmus pro jinou.

Věta „celý svět se netočí kolem USA“ připomíná pestrost návrhů a hodnoticích institucí. Pro bezpečnost však není rozhodující národnost autora. Důležitý je veřejně popsaný návrh, nezávislá analýza, vhodné parametry a kvalitní implementace.

> **Zapamatujte si vztahy:** AES je konkrétní standard vybraný z Rijndaelu; Serpent a Twofish jsou jiní finalisté; Camellia je jiná šifra; NESSIE byl hodnoticí projekt zahrnující více druhů primitiv, nikoli samotný šifrovací algoritmus.

27. Co v prezentaci upřesnit

PŘEHLED OPRAV A PODMÍNEK PRO SAMOSTUDIUM

Snímek	Přesnější výklad	Kapitola
3	Difúze rozptyluje závislosti; přibližně polovina změněných bitů je lavinový efekt v průměru, ne přesná rovnost.	2
4	3DES není obecně 168bitově bezpečné. Stažení standardu a zákaz nové ochrany v rámci NIST nejsou univerzální právní zákaz.	3
5	Stejná Feistelova struktura pro dešifrování vyžaduje správné pořadí podklíčů a konvenci prohození polovin.	4
6	Výběr AES proběhl roku 2000; vydání standardu roku 2001 a účinnost roku 2002.	5
7-8	Blok má vždy 128 bitů. Počáteční AddRoundKey není další plná runda a podklíčů je N_r + 1.	5-12
9	Implementace nemění velikost bloku pevně zvoleného AES. Disková jednotka obsahuje více bloků.	1, 13, 22
9	Číslo sektoru může být vstupem pro XTS tweak; není obecně totožné s CBC IV nebo hotovým tweakem bloku.	22
12	CBC vyžaduje nepředvídatelné IV; pouhá jedinečnost nestačí. IV musí být rovněž autentizováno.	15
12	Změna dat před šifrováním a změna šifrového bloku při přenosu mají odlišné důsledky.	16
12	CBC-MAC je autentizační konstrukce s vlastními podmínkami; prosté CBC šifrování autenticitu nedává.	16
13-14	GCM ověřuje i přidružená data; opakované nonce ohrožuje důvěrnost i autenticitu.	20-21
14-15	Konkrétní implementace rozhoduje o postranních kanálech. Hardware ani název ChaCha20 nezaručují odstranění všech úniků.	24
15	2^{256} je velikost prostoru klíčů. Biclique je konkrétní teoretický výsledek, ne praktické prolomení.	23
15	Groverův algoritmus poskytuje ideální kvadratické zrychlení; AES není vůči němu matematicky imunní.	24
16	Nezdrojované počty hlasů nejsou přímou mírou bezpečnosti ani úplným vysvětlením výběru AES.	5, 25
19-20	Camellia není varianta AES; historické portfolio NESSIE není univerzální současný návod k použití.	26

Opravy slouží k porozumění rozsahu tvrzení. U bezpečnostního výroku se vždy ptejte: jaké má parametry, předpoklady, útočníka a podmínky použití?

Procvičení: AES a jeho konstrukce

ÚLOHY 1-8 · NEJPRVE ŘEŠTE BEZ NÁPOVĚDY

1. Klíč, blok a runda. Kolik bajtů má blok AES-256, kolik jeho základní klíč a kolik 128bitových podklíčů potřebuje? Kolikrát proběhne MixColumns a kolikrát AddRoundKey?

2. Feistelova inverze. Použijte F(R,K) = (R + K) mod 16. Zvolte L = 9, R = 6, K = 3. Spočítejte novou levou a pravou polovinu a pak je pomocí inverzních vztahů obnovte. Čísla v této úloze jsou desítková, ale XOR se provádí po bitech.

3. Nevratná rundová funkce. Proč může být Feistelova síť korektně dešifrovatelná, i když F vrací pro dva různé vstupy stejnou hodnotu? Znamená tato vratnost zároveň bezpečnost?

4. Stav AES. Máme vstupní posloupnost bajtů 00, 01, 02, …, 0f. Napište druhý řádek stavu při číslování řádků 0-3. Jak bude tento řádek vypadat po samotném ShiftRows, tedy bez předchozí substituce?

5. Těleso a běžná čísla. Spočítejte 0x57 ⊕ 0x83 a porovnejte s běžným součtem těchto hodnot. Potom spočítejte 0x02·0x83 v tělese AES. Vysvětlete, kdy se používá XOR s 0x1b.

6. MixColumns. Proč b0 sloupce d4 bf 5d 30 vyjde 04? Doplňte jednotlivé násobky 02·d4 a 03·bf a proveďte XOR. Jaký bezpečnostní účel má tato operace a proč se nesmí používat běžné násobení bajtů?

7. Klíčový rozvrh. AES-128 má jedenáct 128bitových podklíčů. Proč to nedává 1 408 bitů nezávislé klíčové bezpečnosti? Vysvětlete rozdíl mezi expanzí klíče a nezávislým losováním nových klíčů.

8. Inverzní operace. Které operace musíme obrátit při dešifrování AES a která z nich je vlastní inverzí? Proč u CFB nebo CTR používáme pro obnovu zprávy dopředné E_K, přestože chceme „dešifrovat“?

Než přejdete k další straně

Zkuste shrnout jednu rundu AES vlastními slovy bez názvů funkcí: co se nahrazuje, co přesouvá, co míchá a kde se přidává tajemství? Pak jednotlivým krokům přiřaďte správné názvy.

> **Doporučená míra zvládnutí:** rozumět jednomu průchodu a umět ověřit malé výpočty. Memorování celé tabulky S-boxu nebo ruční výpočet všech rund nepřináší srovnatelný užitek.

Procvičení: režimy a bezpečnost

ÚLOHY 9-16

9. ECB a rovnost bloků. Zpráva obsahuje bloky A, B, A, A, C. Co o jejich struktuře prozradí ECB bez znalosti klíče? Odstraní problém přechod z AES-128 na AES-256?

10. Padding. Kolik bajtů přidá PKCS#7 pro CBC-AES ke zprávám o délce 31 a 32 bajtů? Jaké hodnoty mají doplňované bajty v hexadecimálním zápisu?

11. CBC při přenosu. V již zašifrované zprávě se převrátí jediný bit v C3. Které původní bloky budou po dešifrování změněny a jak? Co by se stalo při změně bitu pouze v IV? Předpokládejte zachované hranice a pořadí bloků.

12. Opakování proudu. Dvě zprávy použily v CTR stejný proud. Znáte C1 = 0110, C2 = 1001 a první původní zprávu P1 = 1100. Obnovte proud a druhou původní zprávu. Co bylo porušeno v podmínkách režimu?

13. IV versus nonce. Posuďte tři návrhy: a) CBC používá přímo veřejný rostoucí čítač jako IV; b) CTR po restartu začíná se stejným klíčem a stejným vstupem čítače; c) XTS při přepsání stejného sektoru znovu používá jeho identifikátor. Jsou všechny tři situace stejná chyba?

14. GCM a přidružená data. Do A zahrneme veřejnou hlavičku, do P tajný obsah. Co se stane po změně hlavičky během přenosu? Co když útočník jen pošle celý nezměněný platný záznam podruhé? Jakou další funkci musí řešit protokol?

15. CBC-MAC. Máte známý tag t = E_K(M) jedné plnoblokové zprávy. Proč naivní CBC-MAC s nulovým počátkem vrátí stejný tag pro M || (M ⊕ t)? Co tento příklad ukazuje o přijímání různých délek?

16. Útoky a výběr šifry. Z následujících tvrzení opravte alespoň jednu chybu v každém: „AES-256 zaručuje 256 bitů bezpečnosti i na napadeném zařízení.“ „Hardware eliminuje všechny postranní kanály.“ „Grover dělá AES prakticky okamžitě prolomitelným.“ „Serpent je automaticky bezpečnější, protože má více rund.“

Společný příběh

Navrhněte slovní popis ochrany síťové zprávy: kde vznikne tajný klíč, co bude nonce, co přidružená data, co tag a kdy lze obsah použít? Pak vysvětlete, proč se stejný formát nemusí hodit pro přepis pevně velkého diskového sektoru.

> **Při odpovědích vždy přidejte mechanismus.** Nejen „nonce je špatně“, ale také „opakoval se proud, takže XOR výsledků odstraní masku“. Tak poznáte, že princip umíte použít i v jiném příkladu.

Řešení úloh 1-8

KONSTRUKCE A VÝPOČTY

1. Blok AES-256 má **16 bajtů**, základní klíč **32 bajtů**, rund je 14 a podklíčů **15**. MixColumns proběhne **13krát**, protože v poslední rundě chybí. AddRoundKey proběhne **15krát**, včetně počátečního přidání před rundami.

2. F(6,3) = 9. Nová levá polovina je 6, nová pravá 9 XOR 9 = 0. Inverze: původní R = 6; původní L = 0 XOR F(6,3) = 9. Poloviny se obnoví přesně. Při práci s XOR není možné nahradit operaci obyčejným sčítáním.

3. Pro inverzi se z nové levé poloviny zjistí původní pravá a funkce F se znovu vyhodnotí dopředně. Není třeba obracet F. Tím se ale zaručuje jen korektnost. Například F = 0 vede k prostému prohození polovin, které žádný smysluplný obsah neutají.

4. Druhý řádek je řádek s indexem 1: **01 05 09 0d**. Po posunu o jeden bajt vlevo vznikne **05 09 0d 01**. Stav se plní po sloupcích; kdybychom ho omylem plnili po řádcích, vyšlo by jiné uspořádání.

5. XOR je **0xd4**; běžný součet je **0xda**. Pro xtime(0x83) posuneme bity, omezíme na osm bitů a dostaneme 0x06. Původní nejvyšší bit byl 1, proto provedeme 0x06 XOR 0x1b = **0x1d**. XOR s 0x1b zajišťuje příslušnou polynomiální redukci.

6. 02·d4 = b3 a 03·bf = da. Tedy b3 XOR da XOR 5d XOR 30 = **04**. MixColumns rozptyluje závislost mezi bajty sloupce. Aritmetika je definována nad $GF(2^8)$; běžné celočíselné násobení by bylo jinou transformací a neodpovídalo by AES.

7. Celý rozvrh deterministicky určuje původních 128 bitů. Existuje nejvýše 2^{128} možných rozvrhů pro AES-128, nikoli 2^{1408} nezávisle volitelných kombinací. Expanze přidává strukturované použití klíče uvnitř šifry, ne nové tajemství.

8. Přímé dešifrování AES používá InvShiftRows, InvSubBytes a InvMixColumns a správně seřazené podklíče; AddRoundKey je vlastní inverzí. CTR a CFB ale neobnovují zprávu aplikací D_K na šifrový blok. Vytvoří stejnou masku pomocí E_K a odstraní ji XORem. Dešifrování celého režimu tedy není vždy použití dešifrovací transformace samotné blokové šifry.

> **Shrnutí rundy slovy:** změnit hodnotu každého bajtu, přesunout bajty mezi sloupci, promíchat hodnoty v každém sloupci a XORovat odpovídající podklíč. Poslední runda vynechává míchání sloupců.

Řešení úloh 9-16

REŽIMY A BEZPEČNOSTNÍ PODMÍNKY

9. Výstup bude mít vzor X, Y, X, X, Z, kde X, Y a Z zastupují celé šifrové bloky. Útočník rozpozná rovnost pozic 1, 3 a 4. Větší klíč vlastnost ECB neodstraní; jde o deterministické nezávislé šifrování bloků.

10. Ke 31 bajtům přidáme **jeden bajt 01**, takže vznikne 32 bajtů. Ke 32 bajtům přidáme **šestnáct bajtů 10**, takže vznikne 48 bajtů. Hexadecimální 10 znamená desítkových 16, nikoli deset bajtů.

11. P3 bude obecně rozsáhle změněn a v P4 se převrátí přesně odpovídající bit. P5 a další bloky zůstanou vlivem této samotné chyby nedotčeny. Změna bitu IV změní stejný bit pouze v P1. Bez autentizace může aplikace takto změněná data mylně přijmout.

12. Proud je S = 0110 XOR 1100 = **1010**. Druhá zpráva je 1001 XOR 1010 = **0011**. Použití stejného proudu znamená opakování příslušných čítačových vstupů při stejném klíči. Znalost první zprávy tím odhalí i druhou.

13. a) CBC potřebuje nepředvídatelné IV; veřejný čítač přímo tuto podmínku nesplňuje. b) CTR opakuje proud a tím porušuje základní podmínku. c) U XTS je opakované použití identifikátoru stejné přepisované jednotky součástí modelu. Neznamená to záruku skrytí všech časových změn nebo autentizace, ale nejde automaticky o stejnou chybu jako u CTR či GCM.

14. Změna A při nezměněném tagu má způsobit selhání ověření. Celý znovu odeslaný platný záznam však může mít tag stále platný. Protokol proto potřebuje ochranu proti replay, například ověřování pořadových čísel a evidence již přijatých zpráv.

15. První stav je t. Druhý je E_K((M XOR t) XOR t) = E_K(M) = t. Získali jsme tag nové dvoublokové zprávy ze známého tagu jednoblokové. Naivní CBC-MAC proto nemůže bez dalších úprav bezpečně přijímat obě délky. Standardní CMAC řeší zpracování posledního bloku jinak.

16. Dlouhý klíč nechrání otevřená data ani tajemství přečtená malwarem. Hardware může odstranit některé úniky, nikoli všechny fyzické a protokolové útoky. Grover má ideální kvadratické zrychlení, ale obrovské realizační náklady. Počet rund odlišných konstrukcí není přímá srovnávací míra jejich bezpečnosti.

> **Sebekontrola:** umíte vysvětlit, proč dobře fungující AES může být součástí špatného systému? Pokud ano, dokážete rozlišit bezpečnost primitiva, režimu, implementace a celého protokolu.

Slovníček a značení

RYCHLÁ OPORA PŘI OPAKOVÁNÍ

Pojem	Význam
P, C, K	Otevřený blok, šifrový blok a tajný klíč
E_K, D_K	Dopředná a inverzní transformace blokové šifry
XOR, ⊕	Bitová operace vracející 1 pro dva různé bity
Runda	Jedno přesně definované kolo konstrukce
Podklíč	Klíčový materiál odvozený pro část výpočtu
Feistelova síť	Vratná konstrukce se dvěma polovinami a funkcí F
SP síť	Substitučně-permutační konstrukce
Konfúze	Zastření využitelného vztahu ke klíči
Difúze	Rozptýlení vlivu vstupu mezi části výstupu
Lavinový efekt	Rozsáhlá změna výsledku při malé změně vstupu
GF(2^8)	256prvkové těleso používané pro bajtovou aritmetiku AES
Stav AES	Matice 4 × 4 bajty plněná po sloupcích
SubBytes	Nelineární substituce každého bajtu
ShiftRows	Cyklické posuny řádků stavu
MixColumns	Vratné lineární míchání bajtů sloupce
AddRoundKey	XOR stavu s podklíčem
KeyExpansion	Odvození všech rundových podklíčů
IV, nonce	Pomocné vstupy s podmínkami podle konkrétního režimu
Padding	Výplň pro předepsanou délku a jednoznačné odstranění
ECB / CBC	Nezávislé šifrování bloků / řetězení přes předchozí šifrový blok
CFB / OFB	Zpětná vazba šifrového textu / interního výstupu
CTR	Tvorba klíčového proudu šifrováním čítačových bloků
GCM / GHASH	Autentizovaný režim / jeho tělesová autentizační funkce
MAC / CMAC	Ověřovací kód s klíčem / standardní bloková konstrukce MAC
AAD, tag, AEAD	Přidružená data, ověřovací značka, autentizované šifrování
XTS, tweak	Režim pro úložiště a parametr odlišující pozici
Replay	Opakované předložení dříve platné zprávy
Postranní kanál	Únik přes projevy provedení, například čas či spotřebu
Groverův algoritmus	Kvantové hledání s ideálním kvadratickým zrychlením

Zdroje a odborná opora

ČÁST 1 · AES, HISTORIE A REŽIMY

[P] Ondřej Surý: 7UDZK - Přednáška 2, Moderní symetrické blokové šifry. Dodaná prezentace 7UDZK-2.pdf, 21 stran, University of Ostrava. Určuje rozsah témat. Skripta nejsou autorizovaným přepisem výkladu vyučujícího.

[1] NIST: FIPS 197, Advanced Encryption Standard. Vydáno 2001, aktualizováno 2023. Parametry AES, stav, tělesová aritmetika, rundy a klíčový rozvrh. Kapitoly 4 a 5 standardu obsahují přesnou specifikaci operací.

https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf

[2] NIST: AES Development; Report on the Development of AES. Historie výběru a zdůvodnění rozhodnutí.

https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines/archived-crypto-projects/aes-development

https://csrc.nist.gov/pubs/journal/2001/06/report-on-the-development-of-the-aes/final

[3] EFF: DES Cracker; Distributed.Net & EFF Win RSA's DES Challenge III. Původní informace o hledání klíče DES v letech 1998 a 1999.

https://w2.eff.org/Privacy/Crypto/Crypto_misc/DESCracker/

https://w2.eff.org/Privacy/Crypto/Crypto_misc/DESCracker/HTML/19990119_deschallenge3.html

[4] NIST: NIST to Withdraw Special Publication 800-67 Revision 2, 2023. Rozsah stažení TDEA a rozlišení nové ochrany od operací se starými daty.

https://csrc.nist.gov/News/2023/nist-to-withdraw-sp-800-67-rev-2

[5] NIST: SP 800-38A, 2001. Definice ECB, CBC, CFB, OFB, CTR, podmínky inicializace a závislosti bloků. Historický standard je základem vysvětlení mechanismů, nikoli doporučením používat neautentizované režimy samostatně.

https://csrc.nist.gov/pubs/sp/800/38/a/final

[6] NIST: SP 800-38B, CMAC, aktualizace 2016. Autentizační režim založený na blokové šifře.

https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final

Poznámka k výpočtům: malé Feistelovy a bitové příklady jsou výukové. Mezistavy AES a jeho konečný výstup byly přepočítány; celý výstup byl navíc ověřen nezávislou implementací. Výpočty nejsou důkazem odolnosti algoritmu.

Zdroje a odborná opora

ČÁST 2 · AEAD, ÚLOŽIŠTĚ A ÚTOKY

[7] NIST: SP 800-38D, GCM a GMAC, 2007. Konstrukce AEAD, GHASH a požadavky na IV.

https://csrc.nist.gov/pubs/sp/800/38/d/final

[8] NIST: oznámení revize GCM, 2024; druhá výzva před návrhem revize, 2026. Kontext přehodnocování pravidel včetně délky tagů. Není vydáván za již účinný nový standard.

https://csrc.nist.gov/news/2024/nist-to-revise-sp-80038d-gcm-and-gmac-modes

https://csrc.nist.gov/pubs/sp/800/38/d/r1/2prd

[9] Nir, Langley: RFC 8439, ChaCha20 and Poly1305 for IETF Protocols, 2018. Proudová šifra, autentizace a požadavky konstrukce.

https://www.rfc-editor.org/rfc/rfc8439

[10] Rescorla: RFC 8446, TLS 1.3, 2018. AEAD při ochraně záznamů a podporované sady.

https://www.rfc-editor.org/rfc/rfc8446

[11] WireGuard: Protocol & Cryptography. Oficiální popis používaných primitiv.

https://www.wireguard.com/protocol/

[12] NIST: SP 800-38E, XTS-AES, 2010. Režim pro důvěrnost úložišť, výslovně bez autentizace; odkazuje na IEEE 1619.

https://csrc.nist.gov/pubs/sp/800/38/e/final

[13] Bogdanov, Khovratovich, Rechberger: Biclique Cryptanalysis of the Full AES, 2011. ASIACRYPT 2011, str. 344-371. Konkrétní práce citovaná také ve snímku 15.

https://eprint.iacr.org/2011/449

[14] NIST: Post-Quantum Cryptography FAQ; Practical cost of Grover for AES key recovery, 2024. Výklad rozdílu mezi ideálním zrychlením a reálnými náklady kvantového hledání.

https://csrc.nist.gov/projects/post-quantum-cryptography/faqs

https://csrc.nist.gov/Presentations/2024/practical-cost-of-grover-for-aes-key-recovery

Zdroje a pokrytí prezentace

ČÁST 3 · DALŠÍ ALGORITMY A KONTROLA ROZSAHU

[15] Anderson, Biham, Knudsen: Serpent: A Proposal for the Advanced Encryption Standard. Návrhová specifikace; 128bitový blok, 32 rund, S-boxy a bitslicing.

https://www.cl.cam.ac.uk/ftp/users/rja14/serpent.pdf

[16] Schneier a kol.: Twofish: A 128-Bit Block Cipher, 1998. Původní návrh a popis Feistelovské konstrukce s klíčově závislými S-boxy.

https://www.schneier.com/academic/archives/1998/06/twofish_a_128-bit_bl.html

[17] Matsui, Nakajima, Moriai: RFC 3713, A Description of the Camellia Encryption Algorithm, 2004. Parametry a konstrukce Camellie, informace o jejím zařazení do NESSIE.

https://www.rfc-editor.org/rfc/rfc3713

[18] NTT: Camellia, archiv tiskových zpráv a odkazů NESSIE. Kontext výběru Camellie a historického hodnocení.

https://info.isl.ntt.co.jp/crypt/eng/camellia/news/backnumber.html

Kde najdete témata jednotlivých snímků

Snímky prezentace	Zpracování ve skriptech
1-3	Úvod a kapitoly 1-2; význam komiksu také v kapitole 24
4	Kapitola 3: DES, 3DES a jejich omezení
5	Kapitola 4: Feistelova síť a její inverze
6-8	Kapitoly 5-12: historie, stav, matematika a operace AES
9-11	Kapitoly 13-19 a 22: režimy, padding, vzory a disková data
12	Kapitoly 15-16: CBC, šíření změn a CBC-MAC
13-14	Kapitoly 20-21: GCM a AEAD
15	Kapitoly 23-24: kryptoanalýza, implementace a kvantový pohled
16-18	Kapitoly 5 a 25: výběr AES, Serpent a Twofish
19-20	Kapitola 26: Camellia a NESSIE
21	Procvičení a sebekontrola místo samotné závěrečné výzvy k otázkám

Témata zachovávají rozsah dodané přednášky. Doplněné příklady vysvětlují její zkratky; srovnání režimů a přehled oprav pomáhají nepřenášet podmínky jedné konstrukce na jinou.